AUGUST HACKATHON WORKBOOK

Riyadh Urban Development Hackathon

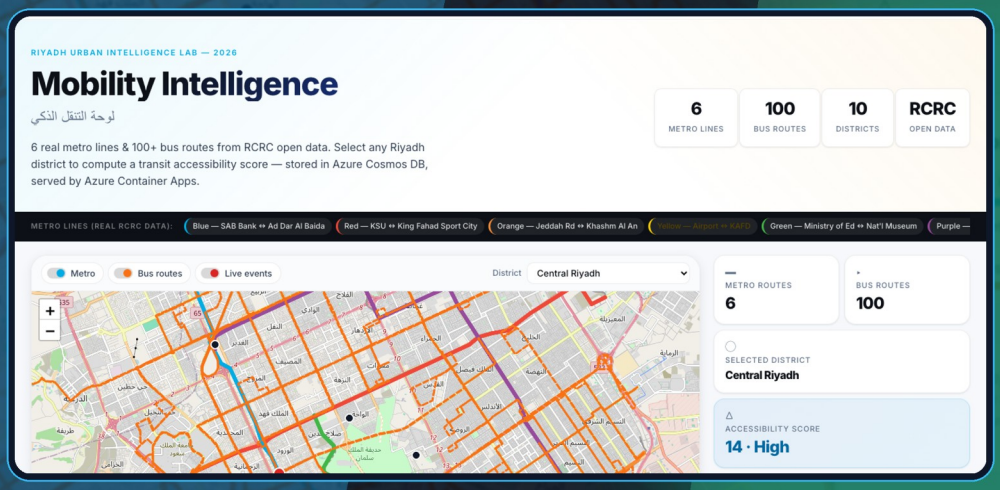
Riyadh Mobility Intelligence Starter Kit

Build and Deployment Workbook

A polished, local-first build path for teams adapting mobility intelligence, district scoring, and Azure-backed urban development prototypes.

Document targets

- **Run locally** — Sample data, map fallback, district selector, score panel
- **Explain the system** — FastAPI, JavaScript, data fallback, Azure services
- **Deploy credibly** — Container Apps, Maps, Blob, Cosmos, App Insights
- **Adapt by track** — Technology, People, Sustainability, Culture



Transformational Technology · Prosperous People · Azure-backed path · Sample-first fallback

Prepared for atomcamp Arabia and Microsoft collaboration

Repository, local app, deployed URL, and Azure resources are listed inside the workbook.

How to Use This Guide

A practical build path for teams who need to understand, run, inspect, deploy, and adapt the Riyadh Mobility Intelligence starter kit with minimal help.

Quick Access

GitHub repo	github.com/Esturban/microsoft-hackathon-riyadh-mobility
Local app	http://127.0.0.1:8000
Deployed app	Paste WEB_APP_URL from azd output here
Resource index	Appendix: Azure docs, deployment notes, project references

Starter Kit, Not Final Model

Use this as a scaffold for credible prototypes. The included score is intentionally simple so builders can replace the data, rules, and Azure services as their track requires.

Local-First Principle

Get the dashboard working from bundled sample data before adding cloud credentials, Blob Storage, Cosmos DB, or a deployed app URL.

Builder Playbook

01 Understand app
Read what the dashboard, layers, score panel, and fallback chain are doing.

02 Run locally
Create the Python environment, start FastAPI, and wait for the slow initial load.

03 Inspect API
Check /health, /api/data-status, routes, districts, and score responses.

04 Select district
Use Riyadh North or Al Olaya and confirm the score panel updates.

05 Deploy Azure
Use az login, azd auth login, and scripts/deploy_azure.sh when ready.

06 Adapt track
Keep the scaffold; replace data, scoring, layers, and Azure services by track.

App layers to keep in view

Backend · Data · Map · Scoring · Cloud-Live

Repository, local app, deployed URL, and Azure references are listed in the workbook appendix.

REPORT BRIEF

Riyadh Mobility Intelligence Starter Kit

Executive Build Brief

A local-first, Azure-ready workbook for teams turning open mobility data into a credible urban intelligence prototype.

106
route records across metro and bus sample layers

10
district records available for scoring and demos

9
API endpoints to inspect, extend, and deploy

7+
Azure services mapped to clear runtime jobs

What this helps teams prove
A working dashboard can connect map layers, district context, explainable scoring, and cloud deployment without requiring every team to start from zero.

What it deliberately does not claim
The score is a teachable proxy, not a formal transport model. Its value is the transparent scaffold: data in, API contract, map state, score explanation, cloud path.

Build Path

01 **Run sample mode**
Confirm local app, map fallback, route layers, district selector, and score panel.

02 **Inspect contracts**
Review /health, /api/data-status, /api/routes, /api/districts, and /api/score.

03 **Connect Azure**
Deploy Container Apps, Maps, Blob, Cosmos, monitoring, and diagnostics.

04 **Adapt for track**
Replace data and scoring while keeping the proven frontend, API, and deployment shape.

Design principle

Keep the first successful path simple: sample data locally, transparent API behavior, then Azure services only when the team is ready to make the prototype cloud-live.

Use this page as the one-minute orientation before the implementation sections.

1. What Builders Are Actually Building

A local-first mobility intelligence scaffold: open the dashboard, inspect transport layers, select a district, and explain an access score.

Core experience

Map the network

Show metro and bus layers with fallback tiles when Azure Maps is not configured.

app/static/src/map.js

Select a district

Anchor every demo around Riyadh district records.

/api/districts

Explain the score

Turn nearby metro, bus, and event inputs into a replaceable score.

app/scoring.py

Starter-kit principle

Run without cloud

Bundled sample data keeps the app useful before Azure is ready.

DATA_MODE=sample

Keep contracts stable

The frontend and API use the same contract locally and in Azure.

FastAPI + static frontend

Swap the domain

Replace data, scoring, or overlays while keeping the app shell.

track adaptation

Judge-facing proof

Visible data layers

Show transit coverage, district context, and live event markers.

dashboard

Transparent logic

The score formula is simple enough to explain live.

/api/score

Azure-ready path

The same containerized app can move to Azure when ready.

azure.yaml + infra/

Build question

Can the city be read?

Can open mobility data become district intelligence quickly?

Transformational Technology

Can access be compared?

Can the score become service access or walkability?

Prosperous People

Can the scaffold extend?

Can the pattern support new urban intelligence tracks?

Sustainable + Culture

Product thesis: prove the end-to-end build path first, then let teams make the domain model more ambitious.

2. Urban Challenge Track Mapping

Lead with Transformational Technology, then use the same district-scoring scaffold for Prosperous People and targeted extension tracks.

Primary story

Transformational Technology

Use mobility layers, route visibility, fallback-aware data, APIs, and Azure deployment to prove a working urban

primary fit

Prosperous People

Convert the district score into service access, walkability, 15-minute city coverage, or quality-of-life comparison.

strong secondary

Builder pitch

Show a credible local app first, then explain how the same scaffold can support richer district intelligence.

demo narrative

Extension paths

Sustainable Solutions

Add AQI, heat, emissions, low-carbon route scoring, and clean-corridor recommendations after the core map

extension path

Culture

Add heritage sites, event-day routing, visitor flows, multilingual notes, and crowd-sensitive access planning.

extension path

What to keep

Keep the frontend shell, FastAPI contracts, sample fallback, score panel, and Azure deployment shape.

starter-kit reuse

Azure emphasis

Maps

Azure Maps supports the cloud map path once keys are configured.

AZURE_MAPS_KEY

Data

Blob Storage and Cosmos DB support processed files and app-shaped records.

Blob + Cosmos

Runtime

Container Apps, App Insights, and Log Analytics make the prototype deployable and observable.

cloud-live

Positioning rule: do not sell this as a final transport model. Sell it as a credible build path for urban intelligence prototypes.

Services and Tools Matrix

The stack reads best as a set of clear jobs: local app runtime, data and maps, Azure hosting, and diagnostics. Each tool has one reason to exist.

Local app runtime

FastAPI
Backend API plus static frontend serving.

`app/main.py, app/routes.py`

Vanilla JavaScript
Dashboard boot, map controls, panels, and API calls.

`app/static/src/`

Sample files
Guaranteed fallback so teams can demo without Azure.

`app/static/sample-data/`

Map and cloud data

Azure Maps
Cloud map SDK and services when a key is configured.

`AZURE_MAPS_KEY`

Blob Storage
Processed GeoJSON and JSON files for cloud data mode.

`upload_to_blob.py`

Cosmos DB
Route, district, and event records for app-shaped data.

`seed_cosmos.py`

Deploy and observe

Container Apps
Hosted FastAPI/frontend runtime.

`azure.yaml, infra/main.bicep`

Container Registry
Container image storage for the deployed app.

`infra/modules/`

App Insights + Logs
Health, failures, latency, requests, and diagnostics.

`Azure portal`

Infrastructure workflow

Bicep
Infrastructure-as-code for reproducible Azure resources.

`infra/main.bicep`

Azure Developer CLI
Environment provisioning and deploy workflow.

`azd / deploy script`

Tests
API, data-shape, health, and scoring checks.

`tests/`

Builder move: explain the services as responsibilities, not vendor names. The judge should hear runtime, map services, cloud data, monitoring, infrastructure, and local fallback.

Architecture Blueprint

The system is designed to work locally first, then graduate to Azure-backed data, maps, and telemetry without changing the builder-facing workflow.

Local-first path

Browser calls FastAPI. FastAPI serves the static frontend and API. The data layer reads bundled sample files so the app still works without Azure credentials.

Azure-backed path

The same app runs in Container Apps, uses Azure Maps when configured, reads Blob and Cosmos data when available, and reports telemetry to Application Insights.

Fallback path

Cosmos DB is preferred when configured, Blob Storage is next, and bundled sample files are always the guaranteed floor for demos and local rebuilds.

Fallback Chain

1. **Cosmos DB** — route, district, event records
2. **Blob Storage** — processed GeoJSON and JSON files
3. **Bundled sample files** — always-available local demo data

Score Request Flow

1. User selects district
2. Frontend calls /api/score
3. Backend loads district/routes/events
4. Formula returns score + reasons
5. Panel renders explanation

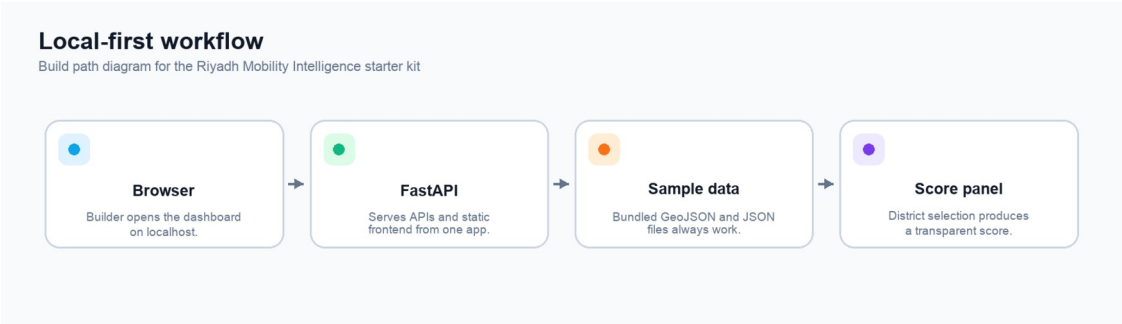
Key implementation rule: Azure should enhance the app, not be required for the first successful rebuild. That constraint keeps hackathon teams moving even when cloud setup is still in progress.

3. Architecture and Build Path

The app is designed to work locally first, then graduate into an Azure-backed live path. The core principle is simple: **the app must remain useful even when Azure credentials, remote services, or live data are unavailable.**

Local-First Workflow

Local mode is the first build milestone. It should work on a builder laptop with no Azure account configured.

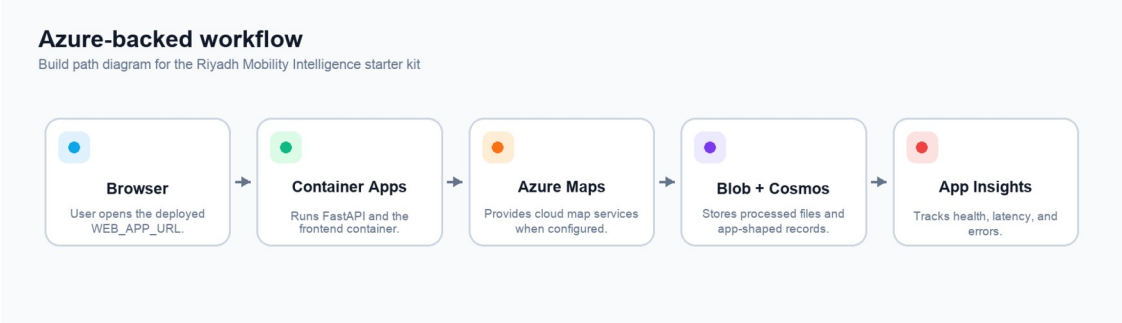


Local-first workflow

Local mode uses the same UI and API shape that the cloud deployment uses. The difference is the data source: bundled sample files replace cloud-backed records.

Azure-Backed Workflow

When deployed, the same containerized app runs in Azure Container Apps and can connect to Azure services for maps, storage, records, and monitoring.

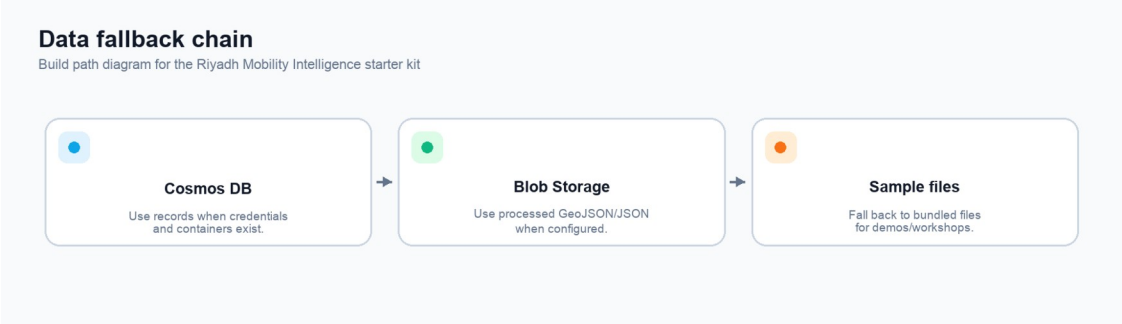


Azure-backed workflow

Use this path when the team is ready to show a credible cloud deployment or run post-deploy smoke tests.

Data Fallback Chain

The app should not fail just because cloud data is missing. Data access is intentionally fallback-aware.



Data fallback chain

In practice, builders should start with `DATA_MODE=sample`, then move to Blob or Cosmos only after the local app is working.

4. Project Structure

Start with the app shell, API, frontend, sample data, deployment files, and tests.

```
riyadh-mobility-intelligence-dashboard/
├── app/
│   ├── main.py                 # FastAPI app shell and static serving
│   ├── routes.py               # API endpoints
│   ├── data_access.py          # sample, Blob, Cosmos data loading
│   ├── scoring.py              # district mobility score
│   ├── azure_clients.py        # Azure SDK client setup
│   ├── config.py               # environment variables and defaults
│   └── static/
│       ├── index.html          # single-page dashboard shell
│       ├── src/                # frontend JavaScript and CSS
│       └── sample-data/        # always-on sample files
├── scripts/                    # fetch, normalize, upload, seed, validate
├── infra/                      # Bicep infrastructure modules
├── docs/                       # build guide and supporting notes
├── tests/                      # scoring, API, and data-shape checks
├── azure.yaml                  # Azure Developer CLI project
├── Dockerfile                  # container definition
└── requirements.txt            # Python dependencies
```

Area	Start here	Why it matters
Backend entrypoints	`app/main.py`, `app/routes.py`	FastAPI app shell, static serving, API contract
Data loading	`app/data_access.py`, `app/static/sample-data/`	sample, Blob, and Cosmos fallback behavior
Score formula	`app/scoring.py`	transparent scoring logic that teams can adapt
Frontend	`app/static/index.html`, `app/static/src/main.js`	page shell, boot sequence, API orchestration
Map behavior	`app/static/src/map.js`, `app/static/src/layers.js`	Azure Maps path, local fallback, overlays
Deployment	`azure.yaml`, `infra/main.bicep`, `scripts/deploy_azure.sh`	cloud-live path
Tests	`tests/`	API, data-shape, health, and

Area	Start here	Why it matters
		scoring checks

Where the Data Comes From

The bundled sample data is derived from Riyadh Commission for Riyadh City open geospatial datasets. The files live under `app/static/sample-data/` and keep the app useful even when no cloud services are configured.

File	Contents	Records
`riyadh_metro_lines_sample.geojson`	Metro line features with names, colors, and geometry	6
`riyadh_bus_routes_sample.geojson`	Bus route features with route IDs and geometry	100
`district_centers_sample.geojson`	Riyadh district center points with English and Arabic names	10
`mock_live_events_sample.json`	Sample delay or incident markers	variable

Fresh data workflow:

```
python3 scripts/fetch_rcrc_data.py
python3 scripts/normalize_to_geojson.py
python3 scripts/validate_data.py
python3 scripts/upload_to_blob.py              # optional cloud step
PYTHONPATH=. python3 scripts/seed_cosmos.py    # optional cloud step
```

Local Run Playbook

The fastest credible path is sample mode first: start the app, wait through slow load, verify the UI, then inspect API diagnostics.

01	Clone	`git clone <repo>`	**Expected signal** folder exists
02	Environment	`python3 -m venv .venv`	**Expected signal** virtual env created
03	Install	`pip install -r requirements.txt`	**Expected signal** FastAPI + test deps installed
04	Configure	`cp .env.example .env`	**Expected signal** DATA_MODE=sample
05	Start	`python -m uvicorn app.main:app --reload`	**Expected signal** server at 127.0.0.1:8000
06	Verify	`/health and /api/data-status`	**Expected signal** ok + sample mode

Slow-load rule

Wait for district selector, layer controls, map area, and score panel. External tiles may lag; overlays and panels are the success signal.

Local success state

Dashboard loads, metro and bus layers appear, selector lists 10 districts, score panel updates, and data status reports sample mode.

Builder move: do not add Azure complexity until this page passes. Local reliability is the foundation for every credible demo, deployment, and track adaptation.

5. Run Locally

Local setup is deliberately direct. The goal is to get one useful screen running before adding cloud services.

Play-by-Play

```
git clone https://github.com/Esturban/microsoft-hackathon-riyadh-mobility.git
cd microsoft-hackathon-riyadh-mobility
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
```

Open `.env` and confirm the default local mode:

```
DATA_MODE=sample
```

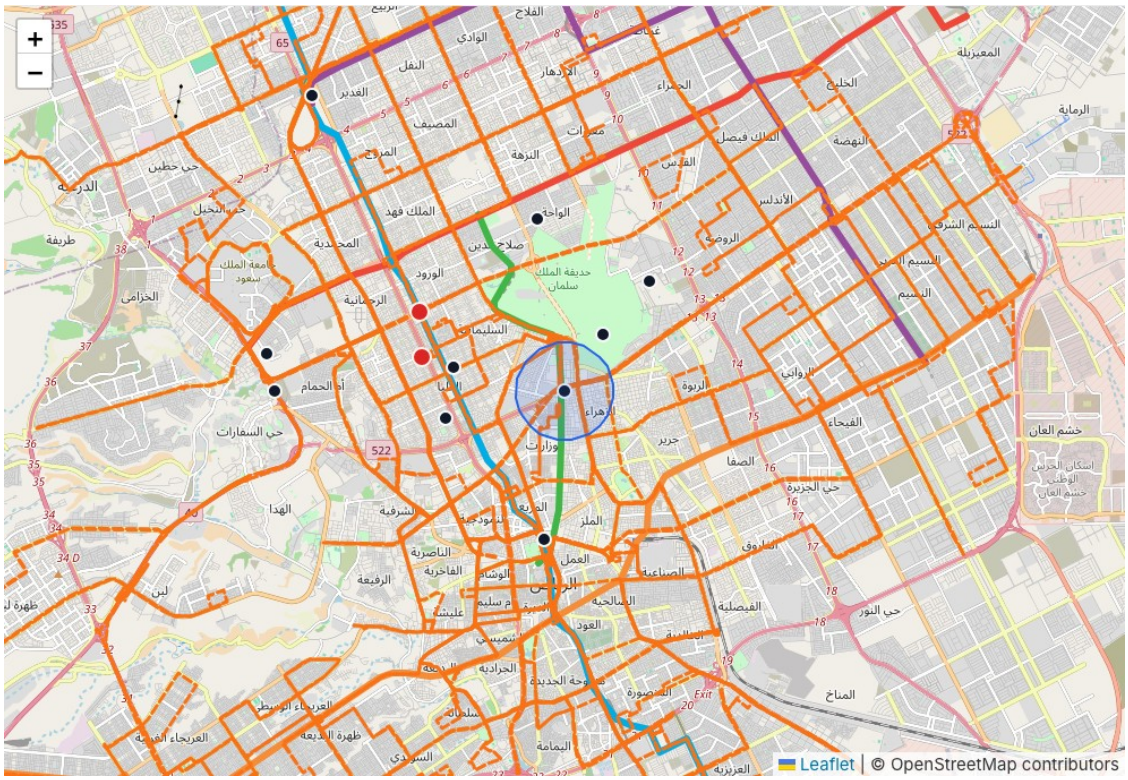
Start the app:

```
python -m uvicorn app.main:app --reload
```

Open the dashboard:

```
http://127.0.0.1:8000
```

The first page load can take time. Wait for the district selector, layer controls, map container, and score panel area before deciding whether the app is loaded. If Azure Maps is not configured, the app should use the OpenStreetMap fallback and still show the route overlays.



Map layers screenshot

Figure: metro and bus layers rendered locally with OpenStreetMap fallback tiles.

Local Verification Gates

Do not judge the first load too early. Wait for the selector, layer controls, map, and score panel, then verify the API and data mode.

UI readiness

Dashboard loads

Open http://127.0.0.1:8000 and wait for the app shell.

local browser

Map renders

Azure Maps or OpenStreetMap should render with route overlays.

map + fallback

Districts appear

The selector should show 10 districts and update the panel.

district selector

API checks

Health endpoint

Return a small OK response from FastAPI.

curl -s /health

Data status

Confirm sample mode, counts, and fallback notes.

curl -s /api/data-status

Score endpoint

Call or trigger the endpoint to confirm scoring.

/api/score?districtId=...

Builder commands

Validate data

Run this when map layers or counts look wrong.

python scripts/validate_data.py

Run tests

Catch API, health, data-shape, and scoring regressions.

python -m pytest

Stay local first

Move to cloud only after the local proof path is stable.

DATA_MODE=sample

Success signal: dashboard visible, overlays present, selector populated, score panel updating, /health OK, and /api/data-status explaining the active data source.

Backend API Contracts

FastAPI is the contract between the dashboard, sample data, optional cloud data, and the score explanation builders show to judges.

Bootstrap

```
/health
/api/config
```

App alive, runtime settings, map mode, data mode.

Map layers

```
/api/routes
/api/routes/geojson?mode=metro
/api/routes/geojson?mode=bus
```

Route summaries plus map-ready metro and bus geometry.

District scoring

```
/api/districts
/api/score?districtId=...
```

District selector records and transparent score components.

Diagnostics

```
/api/live-events
/api/data-status
```

Mock events, active data mode, counts, and fallback status.

Fallback-aware data path

1. Cosmos DB when configured
2. Blob Storage when configured
3. Bundled sample files always

Builder move

Start by reading /api/data-status. It explains whether the app is using sample files, Blob, or Cosmos.

Judge signal

Use /api/score to show that the dashboard is explainable, not just a map with overlays.

Keep API responses boring and reliable. The creativity belongs in the data, scoring, and track adaptation, not in a fragile backend contract.

6. Backend Walkthrough

The backend is a small FastAPI application. It serves both the API and the static frontend so the starter kit stays easy to run and easy to deploy.

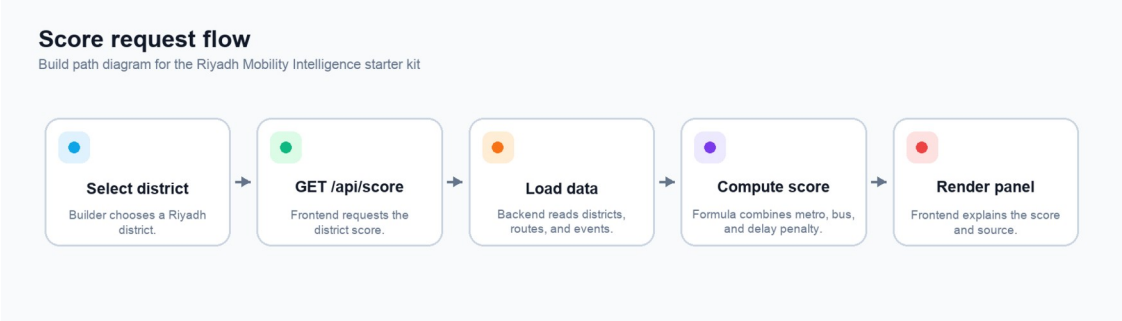
Read the API as four small contracts:

- **Bootstrap:** `/health` and `/api/config` confirm app health and runtime settings.
- **Map layers:** `/api/routes` and `/api/routes/geojson?mode=...` return route counts and map-ready geometry.
- **District scoring:** `/api/districts` and `/api/score?districtId=...` drive the selector and score panel.
- **Diagnostics:** `/api/live-events` and `/api/data-status` explain demo events, active data mode, and fallback behavior.

API data status screenshot

Figure: `/api/data-status` confirms sample mode and explains the fallback path.

Score Request Flow



Score request flow

The score formula stays on the backend so it can be tested and reused:

```
score = (nearby metro count x 3) + nearby bus count − live delay penalty
```

Keep this formula easy to explain. Teams can later replace the weights or add new inputs such as parking, walkability, public services, heat, or event density.

7. Frontend Walkthrough

The frontend is a single-page application built with Vanilla JavaScript. This keeps the starter kit accessible to mixed-experience teams while still showing a complete app workflow.

On startup, `app/static/src/main.js` fetches the core runtime data in parallel:

```
const [config, routes, metro, bus, districts, events, dataStatus] = await Promise.all([
  api.getConfig(),
  api.getRoutes(),
  api.getRouteGeojson("metro"),
  api.getRouteGeojson("bus"),
  api.getDistricts(),
  api.getLiveEvents(),
  api.getDataStatus(),
]);
```

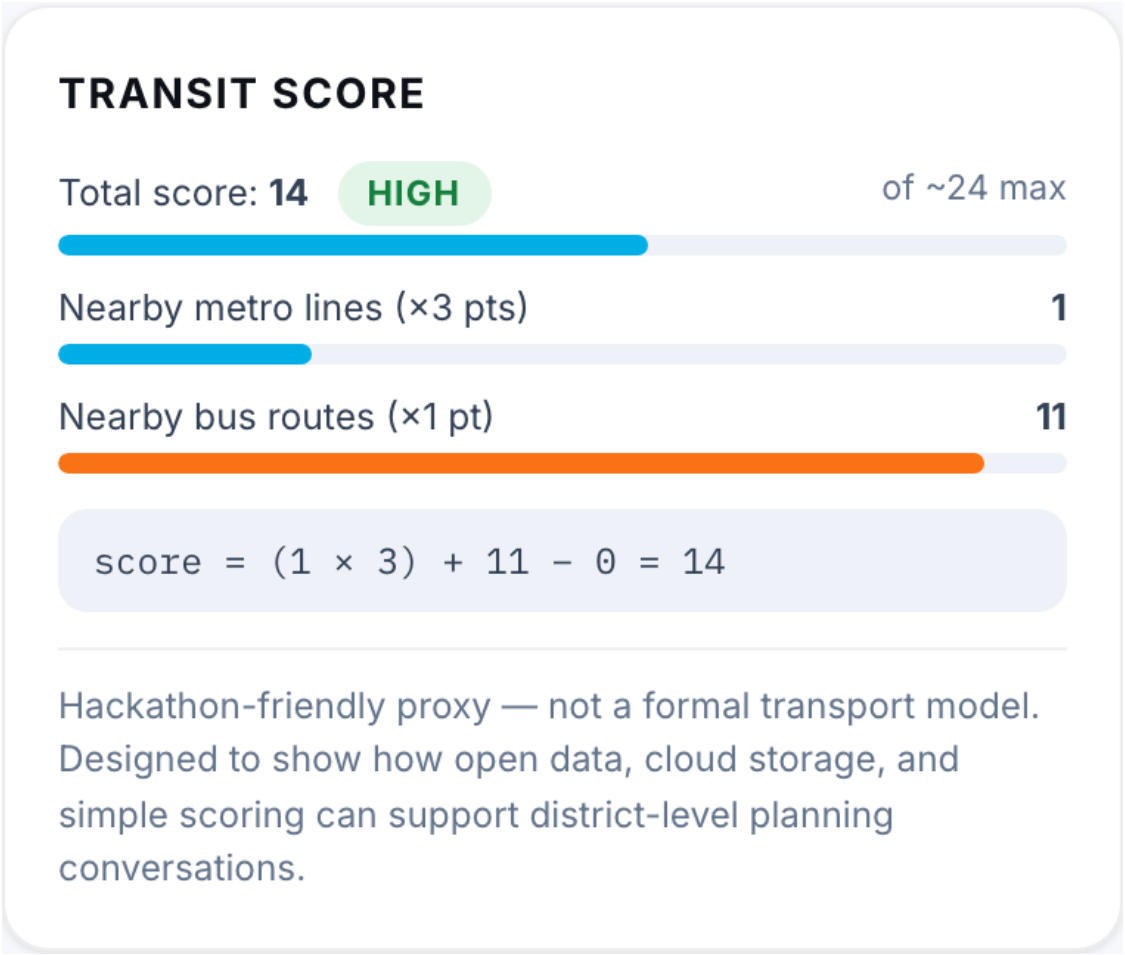
The map layer system makes the city data visible and explainable.

Layer	What it shows	Why it matters
Metro lines	high-capacity mobility corridors	shows structural transit coverage
Bus routes	surface transit coverage	shows fine-grained network reach
Districts	selectable district points	anchors the score conversation
Live events	delay or incident markers	demonstrates cloud-live extension potential
Accessibility buffer	rough selected-district service area	makes the score formula visible

District selector screenshot

The score panel translates raw counts into a judge-friendly story:

- selected district name
- score number
- rating
- nearby metro count
- nearby bus count
- live-event penalty
- readable formula
- active data source context



District score panel screenshot

Figure: district score panel after selecting a Riyadh district.

Cloud Deployment Blueprint

A credible cloud-live path turns the local starter kit into an Azure-hosted app with map services, data storage, monitoring, and a repeatable teardown plan.

1. **Authenticate**
   az login + azd auth login
2. **Initialize**
   azd init and choose subscription/location
3. **Deploy**
   bash scripts/deploy_azure.sh
4. **Verify**
   WEB_APP_URL, /health, /api/data-status
5. **Operate**
   App Insights, logs, fallback status
6. **Teardown**
   destroy_resource_group.sh --yes

Resources Created

- Container Apps
- Container Registry
- Azure Maps
- Blob Storage
- Cosmos DB
- App Insights
- Log Analytics

After-Deploy Checks

- Copy WEB_APP_URL from azd output
- Open dashboard and wait through slow first load
- Confirm /health returns ok
- Confirm /api/data-status reports active mode
- Use App Insights/logs if startup fails

Slow-load expectation

Do not judge success too early. Wait for the district selector, layer controls, map area, and score panel before treating the page as failed.

Cloud-live story for judges

The deployed version uses the same product shape as local mode, then adds Azure Maps, Blob Storage, Cosmos DB, Container Apps, and telemetry so teams can prove a realistic path from starter kit to operational prototype.

8. Cloud-Live Deployment

The cloud-live path is the deployable version of the same starter kit. It is useful when a team needs to show that the app can move beyond a laptop and into a credible Azure environment.

Prerequisites

Before deploying, confirm:

- Azure account is available.
- Azure CLI is installed.
- Azure Developer CLI is installed.
- Shell is authenticated to the right Azure tenant and subscription.
- Target location and resource group are known.
- Local app and tests already pass.

Sign in:

```
az login
azd auth login
azd init
```

Deploy with the helper:

```
bash scripts/deploy_azure.sh
```

Optional location and resource group:

```
bash scripts/deploy_azure.sh eastus rg-riyadh-ud-eastus
```

The helper prints active environment values, then runs the Azure Developer CLI deployment workflow.

What Deployment Creates

Azure resource	Role
Azure Container Apps	hosts FastAPI and the static frontend
Azure Container Registry	stores the built container image
Azure Maps account	supports cloud map services
Azure Blob Storage	stores raw and processed mobility files
Azure Cosmos DB	stores route, district, and event records
Application Insights	tracks health, latency, failures, and requests
Log Analytics	stores logs and diagnostics

After deployment:

1. Copy `WEB_APP_URL` from `azd` output.
2. Open the deployed app in a browser.
3. Check `<WEB_APP_URL>/health`.
4. Check `<WEB_APP_URL>/api/data-status`.
5. Confirm the active data mode is expected.
6. Upload processed files to Blob if using Blob mode.
7. Seed Cosmos DB if using Cosmos mode.

8. Inspect App Insights if the deployed app fails or loads slowly.

Optional cloud data steps:

```
python3 scripts/upload_to_blob.py
PYTHONPATH=. python3 scripts/seed_cosmos.py
```

Teardown when finished:

```
bash scripts/destroy_resource_group.sh --yes
```

The teardown script deletes the current Azure resource group and intentionally requires `--yes`.

Track Adaptation Routes

Keep the proven shell. Replace the domain data, score logic, explanations, and Azure services to make the track-specific intelligence obvious.

Transformational Technology

Mobility Command View

Reuse from starter kit
Keep: route layers, event overlay, district selector, KPIs.

Make it track-specific
Add: live status, parking demand, peak-load simulation.

Prosperous People

District Intelligence

Reuse from starter kit
Keep: district selector, score panel, Cosmos-ready records.

Make it track-specific
Add: clinics, schools, parks, service-access scoring.

Sustainable Solutions

Sustainable Mobility Overlay

Reuse from starter kit
Keep: map shell, buffer logic, fallback data path.

Make it track-specific
Add: AQI, heat, emissions, clean corridor scoring.

Culture

Visitor Movement

Reuse from starter kit
Keep: map interaction, route overlays, selected-area storytelling.

Make it track-specific
Add: heritage markers, event-day routing, multilingual notes.

Winning pattern: do not rebuild the app. Preserve the local-first API, map shell, fallback chain, and deployment path; make the chosen track unmistakable through data and scoring.

9. Starter Kit Adaptation Routes

Treat this codebase as a scaffold. Teams should keep the working shell, replace the domain data, and adapt the score or map layers toward the track they are pursuing.

Route 1: Mobility Command View

Best fit: Transformational Technology.

Keep:

- route layers
- event overlay
- district selector
- route KPIs
- data-status debugging

Replace or add:

- real congestion or delay source
- parking demand layer
- peak-load simulation
- route delay explanations

Likely Azure services: Azure Maps, Container Apps, Event Hubs, Stream Analytics, Cosmos DB, Application Insights.

Route 2: District Intelligence

Best fit: Prosperous People.

Keep:

- district selector
- scoring panel
- map focus behavior
- Cosmos-ready district records

Replace or add:

- service access layers such as clinics, schools, parks, or transit stops
- 15-minute city score
- walkability indicators
- district comparison panel

Likely Azure services: Azure Maps, Blob Storage, Cosmos DB, Container Apps, optional Azure OpenAI for plain-language score explanations.

Route 3: Sustainable Mobility Overlay

Best fit: Sustainable Solutions.

Keep:

- mobility layers
- selected district context
- route scoring pattern
- fallback-aware sample files

Replace or add:

- AQI stations
- heat layer
- pollution hotspot markers
- clean corridor recommendations
- emissions or low-carbon route scoring

Likely Azure services: Azure Maps, Blob Storage, Cosmos DB, Power BI, Application Insights.

Route 4: Culture and Visitor Movement

Best fit: Culture.

Keep:

- map layer structure
- event markers
- route overlays
- selected place or district panel

Replace or add:

- heritage site markers
- crowd-sensitive route suggestions
- multilingual visitor notes
- event-day access planning

Likely Azure services: Azure Maps, Cosmos DB, Blob Storage, Container Apps, optional translation services.

10. Demo Flow

Use this short flow when presenting to mentors or judges.

Step	Show	Explain
1	Dashboard landing view	This is a Riyadh mobility intelligence starter kit, not just a static map
2	Metro and bus toggles	The map layers show public transport coverage
3	District selector	District context drives the scoring workflow
4	Score panel	The score is transparent and easy to adapt
5	`/api/data-status`	The app is fallback-aware and cloud-live ready

Step	Show	Explain
6	Azure workflow diagram	Services map to hosting, maps, files, records, and monitoring
7	Adaptation routes	The same scaffold supports multiple hackathon tracks

Keep the live demo local unless the deployed app has already passed `/health` and `/api/data-status` checks.

Appendix

Common Commands

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python -m uvicorn app.main:app --reload
python -m pytest
python scripts/validate_data.py
bash scripts/deploy_azure.sh
bash scripts/destroy_resource_group.sh --yes
```

Debugging Checklist

- If the page loads slowly, wait for the district selector, layer controls, map area, and score panel before judging success.
- If the map is blank, clear `AZURE_MAPS_KEY` in `.env` and reload so the local fallback can be used.
- If cloud data is missing, open `/api/data-status` first.
- If sample files fail, run `python scripts/validate_data.py`.
- If the deployed app fails, check `/health`, `/api/data-status`, Container App logs, and Application Insights.

Visual Asset Checklist

Asset	Included in this guide
app layer icon row	`docs/assets/rebuild-guide/icon-row-app-layers.png`
four-track icon row	`docs/assets/rebuild-guide/icon-row-tracks.png`
local dashboard screenshot	`docs/assets/rebuild-guide/local-dashboard.png`
map layer screenshot	`docs/assets/rebuild-guide/map-layers.png`
district selector screenshot	`docs/assets/rebuild-guide/district-selector.png`
district score panel screenshot	`docs/assets/rebuild-guide/district-score-panel.png`
data-status endpoint screenshot	`docs/assets/rebuild-guide/api-data-status.png`
local workflow diagram	`docs/assets/rebuild-guide/diagram-local-first.png`
Azure workflow diagram	`docs/assets/rebuild-guide/diagram-azure-backed.png`
fallback chain diagram	`docs/assets/rebuild-guide/diagram-data-fallback.png`
score request diagram	`docs/assets/rebuild-guide/diagram-score-request.png`

Resource Index

Use this page as the resource hub instead of repeating links in the footer. Keep the cover page focused on the repo, local app, and deployed app URL; keep this appendix focused on build, deployment, and Azure references.

Need	Open
GitHub repo	github.com/Esturban/microsoft-hackathon-riyadh-mobility
Local dashboard	http://127.0.0.1:8000
atomcamp Arabia	atomcamparabia.com
Azure Developer CLI docs	Microsoft Learn
`azd up` workflow	Microsoft Learn
Azure Container Apps docs	Microsoft Learn
Azure Maps docs	Microsoft Learn
Azure Cosmos DB docs	Microsoft Learn
Student quickstart	`README.md`
Architecture notes	`docs/architecture.md`
Azure deployment notes	`docs/azure_deployment.md`
Data source notes	`docs/data_sources.md`
Troubleshooting	`docs/troubleshooting.md`
Demo script	`docs/judging_demo_script.md`

Glossary

Azure-backed live path
The deployable path where the starter kit uses Azure services for maps, hosting, files, records, and monitoring.

Blob Storage
Azure file storage for raw and processed mobility datasets.

Cosmos DB
Azure document database for route summaries, district records, and event records.

FastAPI
Python web framework used to serve the API and static frontend.

GeoJSON
JSON format for map features such as points, lines, and polygons.

Mobility Intelligence Starter Kit
The reusable scaffold that teams can adapt into mobility, district, sustainability, or culture prototypes.

Sample fallback
The local files that keep the app useful even when cloud services or remote data are unavailable.

Single-page application
The current frontend shape: one main web page that loads data and updates panels dynamically.

Closing Note

The starter kit is strongest when it stays practical: one useful local screen, clear route layers, transparent scoring, and a cloud-live path that is credible without becoming heavy. Build the smallest version that can be explained well, then extend it toward the hackathon track that matters most.